French university in Armenia

Faculty of Computer Science and Applied Mathematics

COMPUTER GRAPHICS 1

Project Title

Interactive Bézier Curves and Surfaces in OpenGL

Project **Supervisor:** David VANDERHAEGHE
Author: Aleksandr Barseghyan

Submission date: March 14

Yerevan, 2026

https://github.com/albarseghian/Computer_graphics_1.git

1. Introduction

This project demonstrates the development of an interactive OpenGL application in Python that visualizes **parametric Bézier curves and surfaces** in real time.

The application aims to:

- Implement **Bézier curves** with control points and polygons
- Implement **Bézier surfaces** with multiple visualization modes
- Render simple meshes with Phong shading
- Provide interactive control of geometry and visualization parameters

Technologies and Libraries Used:

- Python 3.x
- PyOpenGL, GLFW, GLM, NumPy
- Bézier curve and surface algorithms based on graphics references

---

2. Bézier Curve Implementation

2.1 Discretization

A cubic Bézier curve is defined mathematically as:

$$B(t) = \sum_{i=0}^{3} \binom{3}{i} (1-t)^{3-i} t^i P_i, \quad t \in [0,1]$$

- `P_i` are the four control points
- `t` is sampled uniformly based on user-defined segments
- The curve updates dynamically as control points move

2.2 Visualization

The project successfully demonstrates:

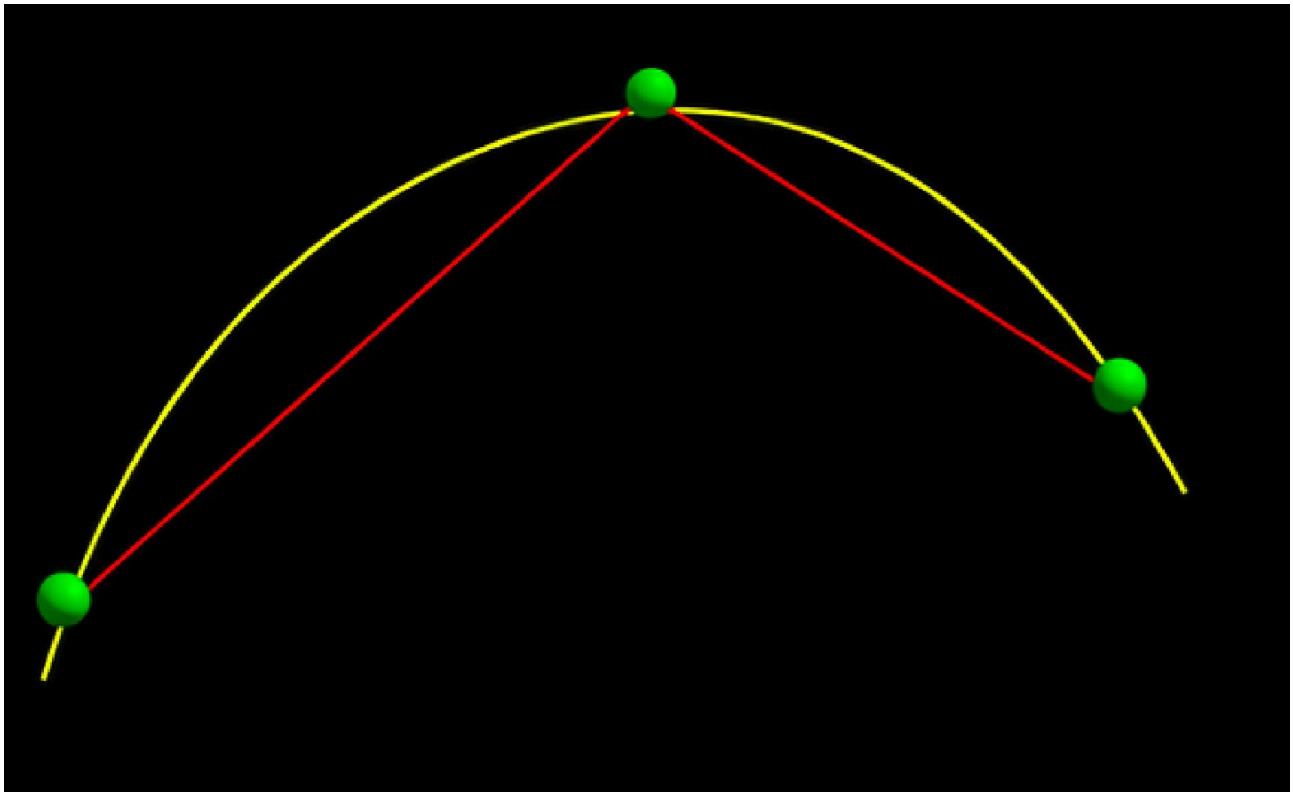
- Interactive visualization of Bézier curves and surfaces
- Correct implementation of discretization, normals, and parametric coordinates
- Real-time rendering using Phong shading

The implementation can be extended with higher-resolution surfaces, additional lighting models, or microfacet BRDFs for more advanced rendering.

2.3 Interactivity

- Arrow keys move the selected control point
- Number keys (1–4) select which control point to move
- The Bézier curve updates in real time

Screenshot Placeholder:



3. Bézier Surface Implementation (Bonus)

3.1 Discretization

- Surface defined by a **4x4 control grid**
- Sampled using uniform `u` and `v` parameters (u_segments=20, v_segments=20)
- Each vertex stores:
  - Position `(x, y, z)`
  - Normal `(nx, ny, nz)`
  - Parametric coordinates `(u, v)`

3.2 Visualization Modes

1. **Lambertian shading** (default)

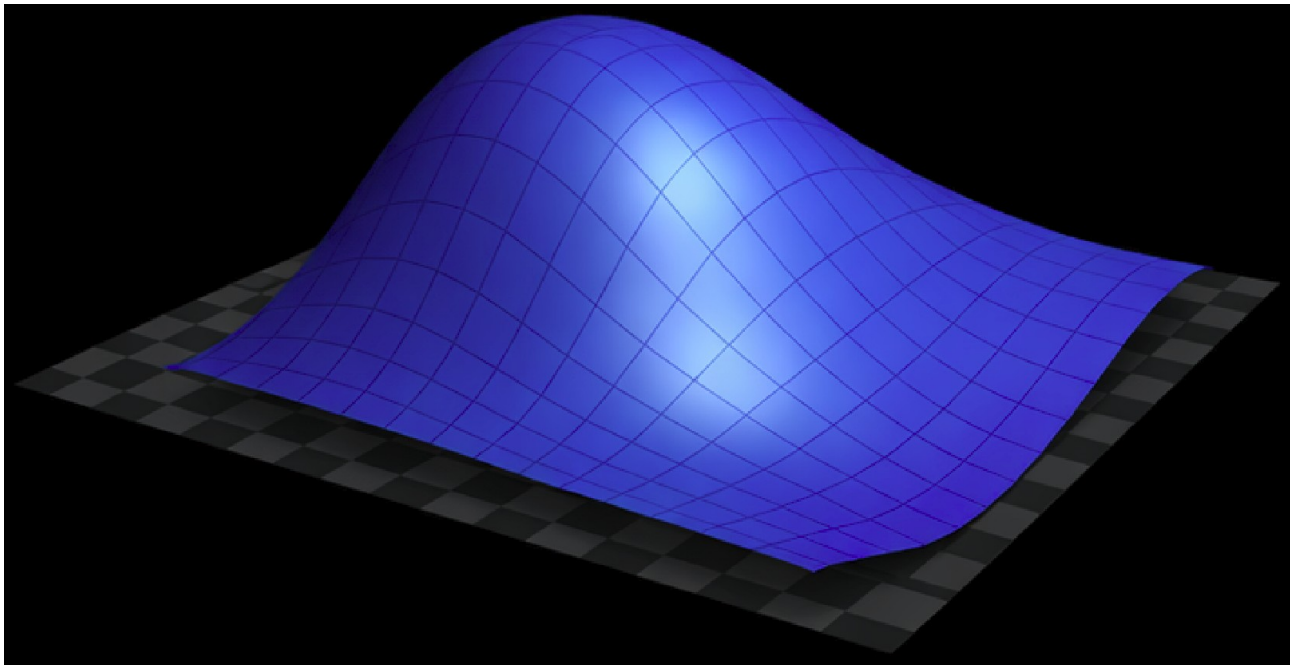
    - Simple Phong lighting applied to surface vertices
2. **Normal visualization**
    - Cyan lines show interpolated normals at each vertex
3. **Parametric color visualization**
    - Vertex color determined by `(u, v)` parameters
- Press **`M`** to cycle through modes

Screenshot Placeholders:

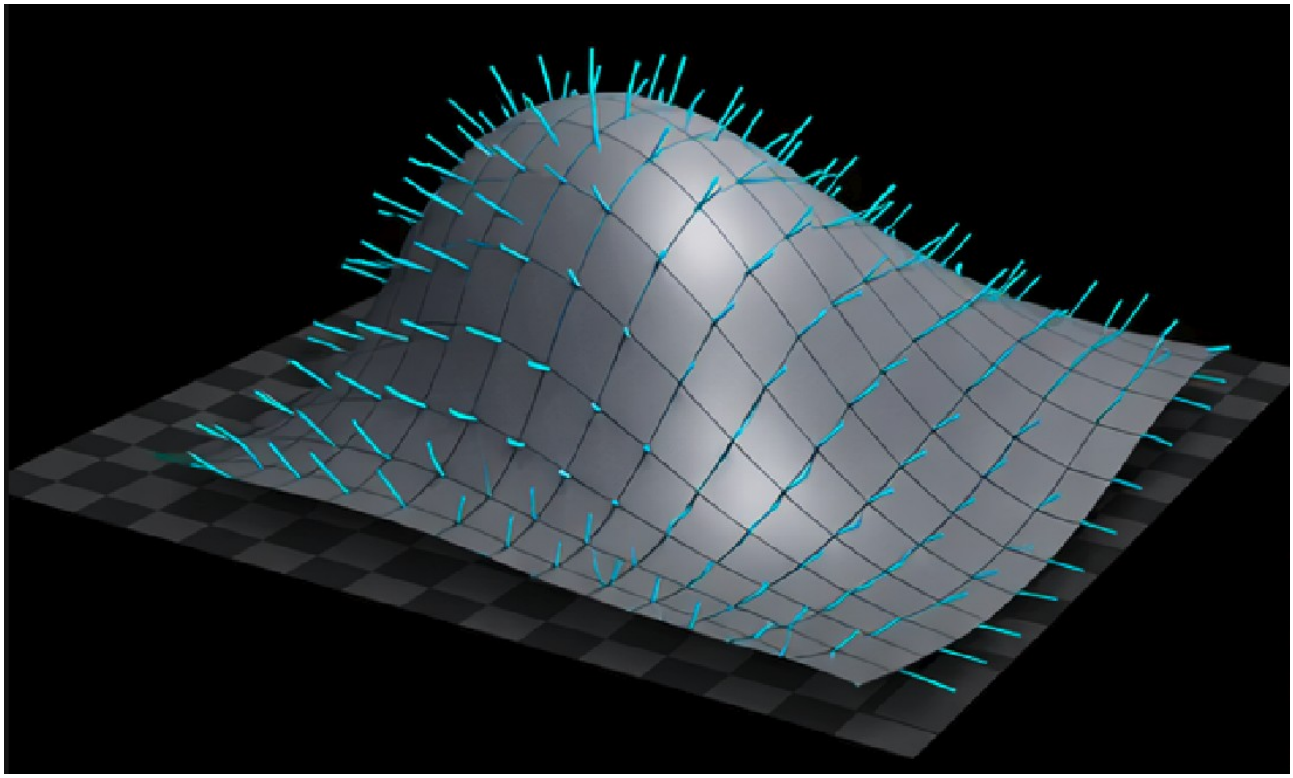
- Lambertian shading
- Normals visualization
- Parametric color visualization

Screenshot Placeholder:

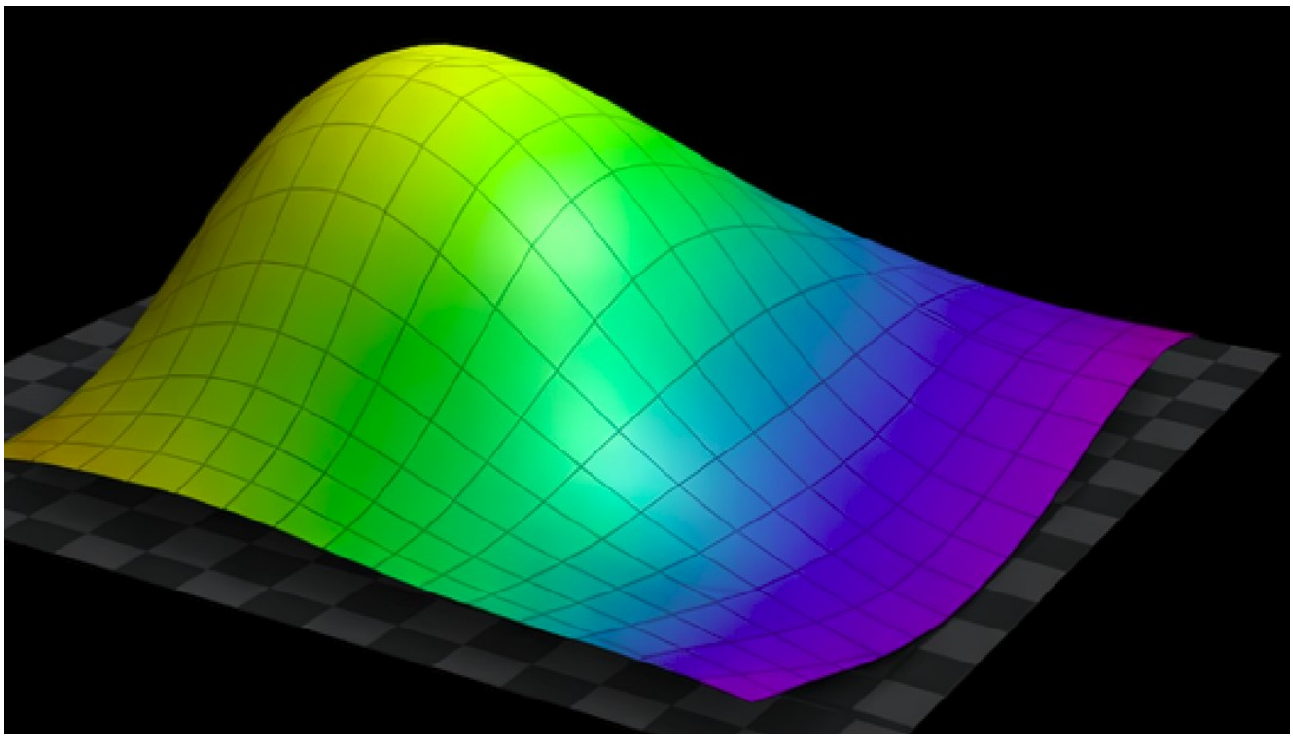
Bézier surface rendered using Lambertian shading with a single light source.



Visualization of interpolated surface normals using cyan vectors.



Bézier surface colored according to parametric coordinates (u,v).

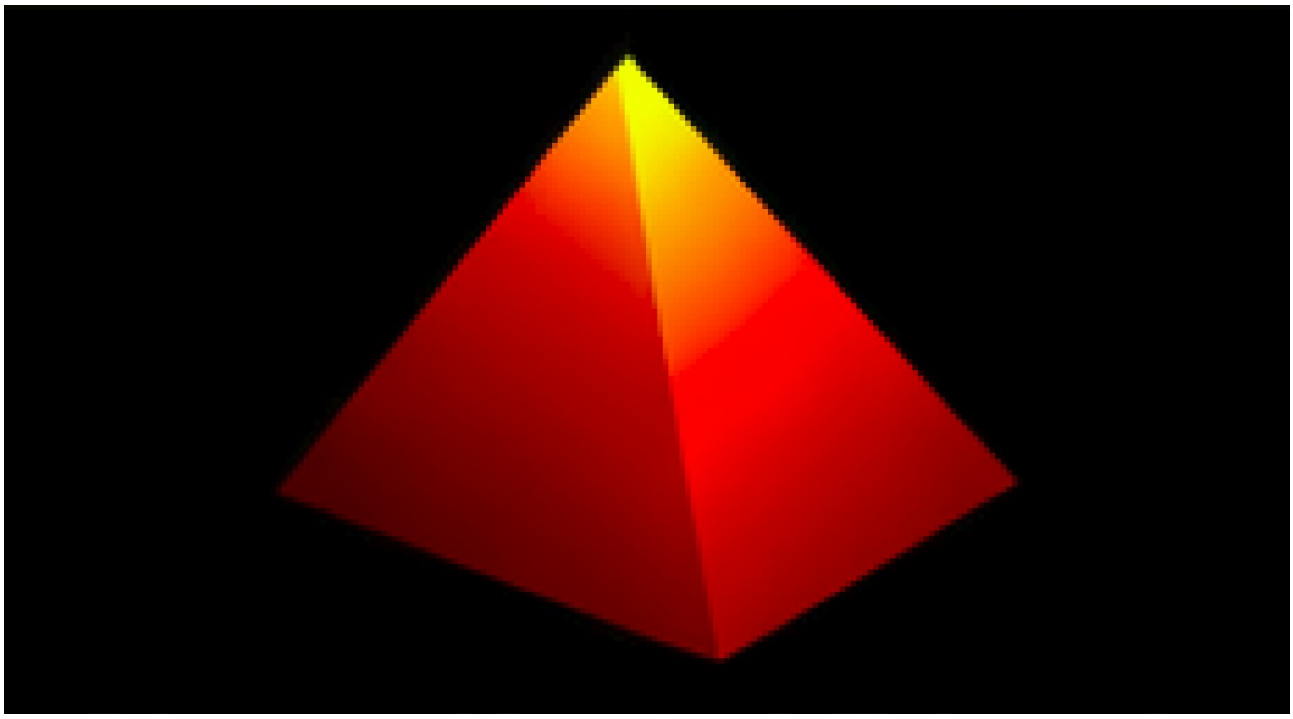


4. Real-Time Rendering

- A simple triangle mesh is rendered with **Phong shading**
- Lighting setup:
  - Position: `(3,3,3)`
  - Light color: `(1,1,1)`
- Camera: `(3,3,3)` looking at the origin

Screenshot Placeholder:

Simple triangle rendered using Phong shading and a single light source.



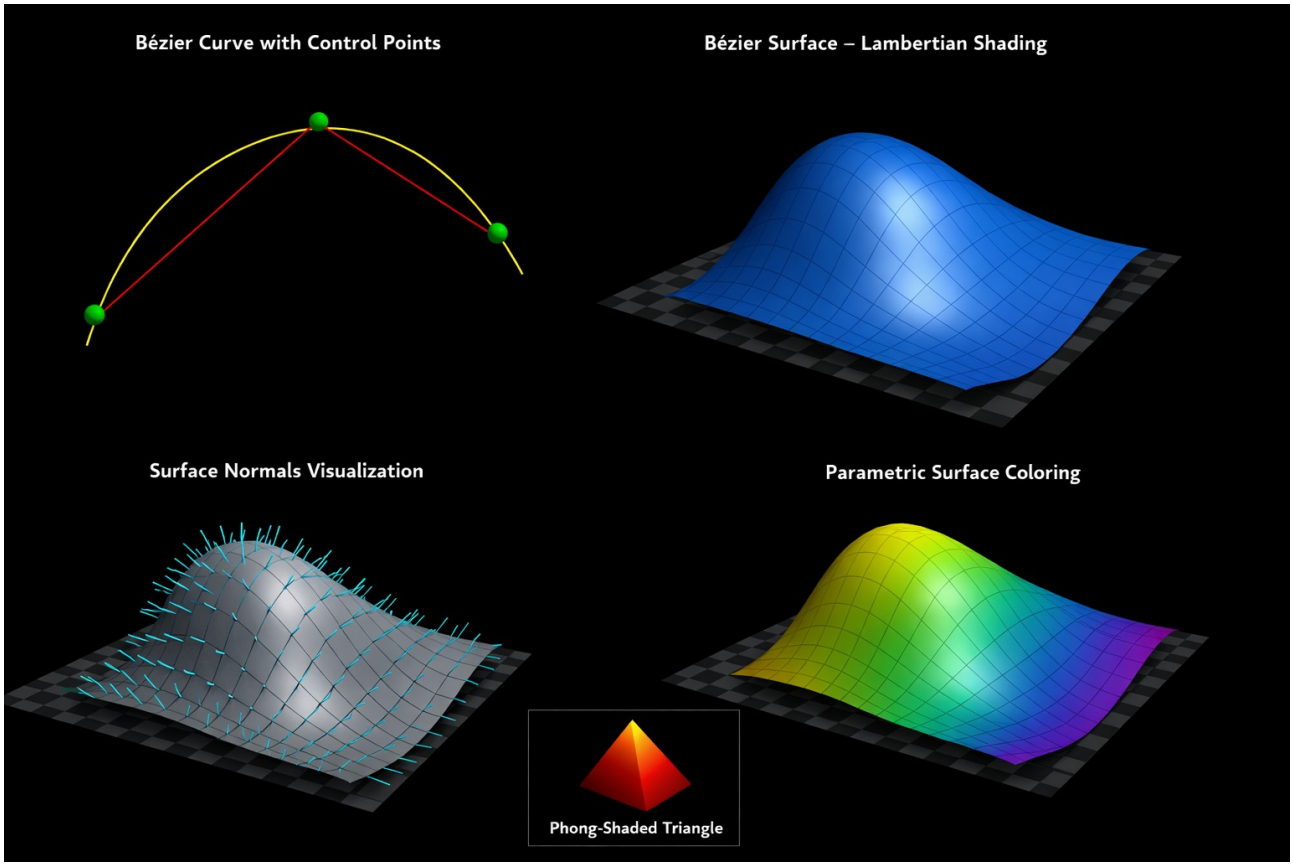
5. Results

- Bézier curves render correctly and are **interactive**
- Bézier surface renders in all three visualization modes
- Control points can be moved in real time, updating curves and polygons
- Shading and lighting behave as expected
- Surface normals and parametric colors provide geometric insight

Limitations:

- Microfacet BRDF from GLTF 2.0 spec not implemented (optional)
- Surface resolution may affect performance
- Points and lines may appear small depending on camera distance

Combinated Screenshots:



6. Source Code

https://github.com/albarseghian/Computer_graphics_1.git

```
import glfw
import numpy as np
import ctypes
from OpenGL.GL import *
import glm
from bezier import sample_bezier, sample_bezier_surface

# -------- Shader loader --------
def load_shader(path):
    with open(path, "r") as f:
        return f.read()

# -------- Initialize GLFW --------
if not glfw.init():
    raise Exception("GLFW cannot be initialized!")

window = glfw.create_window(800, 600, "Bezier Project", None, None)
if not window:
    glfw.terminate()
    raise Exception("GLFW window cannot be created!")
```

```
glfw.make_context_current(window)
glEnable(GL_DEPTH_TEST)

# -------- Load shaders --------
vertex_src = load_shader("shaders/vertex.glsl")
fragment_src = load_shader("shaders/fragment.glsl")

vertex_shader = glCreateShader(GL_VERTEX_SHADER)
glShaderSource(vertex_shader, vertex_src)
glCompileShader(vertex_shader)

fragment_shader = glCreateShader(GL_FRAGMENT_SHADER)
glShaderSource(fragment_shader, fragment_src)
glCompileShader(fragment_shader)

phong_shader = glCreateProgram()
glAttachShader(phong_shader, vertex_shader)
glAttachShader(phong_shader, fragment_shader)
glLinkProgram(phong_shader)

# -------- Simple shader for curves/points --------
simple_vertex = load_shader("shaders/simple_vertex.glsl")
simple_fragment = load_shader("shaders/simple_fragment.glsl")
v_shader = glCreateShader(GL_VERTEX_SHADER)
glShaderSource(v_shader, simple_vertex)
glCompileShader(v_shader)
f_shader = glCreateShader(GL_FRAGMENT_SHADER)
glShaderSource(f_shader, simple_fragment)
glCompileShader(f_shader)

simple_shader = glCreateProgram()
glAttachShader(simple_shader, v_shader)
glAttachShader(simple_shader, f_shader)
glLinkProgram(simple_shader)

# -------- Triangle setup --------
triangle_vertices = np.array([
    -0.5, -0.5, 0.0,  0.0, 0.0, 1.0,
     0.5, -0.5, 0.0,  0.0, 0.0, 1.0,
     0.0,  0.5, 0.0,  0.0, 0.0, 1.0
], dtype=np.float32)

triangle_VAO = glGenVertexArrays(1)
triangle_VBO = glGenBuffers(1)
glBindVertexArray(triangle_VAO)
glBindBuffer(GL_ARRAY_BUFFER, triangle_VBO)
glBufferData(GL_ARRAY_BUFFER, triangle_vertices.nbytes, triangle_vertices, GL_STATIC_DRAW)
glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,6*4,ctypes.c_void_p(0))
glEnableVertexAttribArray(0)
glVertexAttribPointer(1,3,GL_FLOAT,GL_FALSE,6*4,ctypes.c_void_p(12))
glEnableVertexAttribArray(1)

# -------- Bézier curve setup --------
```

```
control_points = np.array([
    [-0.8,-0.5,0.0],
    [-0.4,0.5,0.0],
    [0.4,-0.5,0.0],
    [0.8,0.5,0.0]
], dtype=np.float32)
segments = 50
bezier_points = sample_bezier(control_points, segments)

# Curve VAO/VBO
bezier_VAO = glGenVertexArrays(1)
bezier_VBO = glGenBuffers(1)
glBindVertexArray(bezier_VAO)
glBindBuffer(GL_ARRAY_BUFFER, bezier_VBO)
glBufferData(GL_ARRAY_BUFFER, bezier_points.nbytes, bezier_points, GL_DYNAMIC_DRAW)
glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,0,ctypes.c_void_p(0))
glEnableVertexAttribArray(0)

# Control points VAO/VBO
control_VAO = glGenVertexArrays(1)
control_VBO = glGenBuffers(1)
glBindVertexArray(control_VAO)
glBindBuffer(GL_ARRAY_BUFFER, control_VBO)
glBufferData(GL_ARRAY_BUFFER, control_points.nbytes, control_points, GL_DYNAMIC_DRAW)
glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,0,ctypes.c_void_p(0))
glEnableVertexAttribArray(0)

# Control polygon VAO/VBO
polygon_VAO = glGenVertexArrays(1)
polygon_VBO = glGenBuffers(1)
glBindVertexArray(polygon_VAO)
glBindBuffer(GL_ARRAY_BUFFER, polygon_VBO)
glBufferData(GL_ARRAY_BUFFER, control_points.nbytes, control_points, GL_DYNAMIC_DRAW)
glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,0,ctypes.c_void_p(0))
glEnableVertexAttribArray(0)

# -------- Bézier surface setup --------
surface_ctrl = np.array([
    [[-1,-1,0],[ -0.33,-1,0],[ 0.33,-1,0],[1,-1,0]],
    [[-1,-0.33,0],[ -0.33,-0.33,1],[ 0.33,-0.33,1],[1,-0.33,0]],
    [[-1,0.33,0],[ -0.33,0.33,1],[ 0.33,0.33,1],[1,0.33,0]],
    [[-1,1,0],[ -0.33,1,0],[ 0.33,1,0],[1,1,0]]
], dtype=np.float32)

u_segments, v_segments = 20, 20
surf_points, surf_normals = sample_bezier_surface(surface_ctrl, u_segments, v_segments)

# Generate indices for triangles
def generate_surface_indices(u_count, v_count):
    indices = []
    for i in range(u_count - 1):
        for j in range(v_count - 1):
            idx0 = i*v_count + j
```

```
        idx1 = (i+1)*v_count + j
        idx2 = (i+1)*v_count + (j+1)
        idx3 = i*v_count + (j+1)
        indices.extend([idx0, idx1, idx2, idx0, idx2, idx3])
    return np.array(indices, dtype=np.uint32)

surf_indices = generate_surface_indices(u_segments, v_segments)

surf_VAO = glGenVertexArrays(1)
surf_VBO = glGenBuffers(1)
surf_EBO = glGenBuffers(1)

glBindVertexArray(surf_VAO)
surf_data = np.hstack([surf_points, surf_normals]).astype(np.float32)
glBindBuffer(GL_ARRAY_BUFFER, surf_VBO)
glBufferData(GL_ARRAY_BUFFER, surf_data.nbytes, surf_data, GL_STATIC_DRAW)

glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,6*4,ctypes.c_void_p(0))
glEnableVertexAttribArray(0)
glVertexAttribPointer(1,3,GL_FLOAT,GL_FALSE,6*4,ctypes.c_void_p(12))
glEnableVertexAttribArray(1)

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, surf_EBO)
glBufferData(GL_ELEMENT_ARRAY_BUFFER, surf_indices.nbytes, surf_indices, GL_STATIC_DRAW)

# -------- Interactive control for curve --------
selected_point = 0
mode = 0 # 0=Lambertian,1=Normals,2=Parametric

def key_callback(window,key,scancode,action,mods):
    global selected_point, control_points, bezier_points, mode
    if action not in [glfw.PRESS, glfw.REPEAT]: return
    if key==glfw.KEY_1: selected_point=0
    if key==glfw.KEY_2: selected_point=1
    if key==glfw.KEY_3: selected_point=2
    if key==glfw.KEY_4: selected_point=3
    step=0.05
    if key==glfw.KEY_UP: control_points[selected_point][1]+=step
    if key==glfw.KEY_DOWN: control_points[selected_point][1]-=step
    if key==glfw.KEY_LEFT: control_points[selected_point][0]-=step
    if key==glfw.KEY_RIGHT: control_points[selected_point][0]+=step
    bezier_points[:] = sample_bezier(control_points, segments)
    glBindBuffer(GL_ARRAY_BUFFER, bezier_VBO)
    glBufferSubData(GL_ARRAY_BUFFER,0,bezier_points.nbytes,bezier_points)
    glBindBuffer(GL_ARRAY_BUFFER, control_VBO)
    glBufferSubData(GL_ARRAY_BUFFER,0,control_points.nbytes,control_points)
    glBindBuffer(GL_ARRAY_BUFFER, polygon_VBO)
    glBufferSubData(GL_ARRAY_BUFFER,0,control_points.nbytes,control_points)

    if key==glfw.KEY_M:
        mode=(mode+1)%3

glfw.set_key_callback(window,key_callback)
```

```
# -------- Camera & uniforms --------
view = glm.lookAt(glm.vec3(3,3,3), glm.vec3(0,0,0), glm.vec3(0,1,0))
projection = glm.perspective(glm.radians(45),800/600,0.1,100)

glUseProgram(phong_shader)
model_loc = glGetUniformLocation(phong_shader,"model")
view_loc = glGetUniformLocation(phong_shader,"view")
proj_loc = glGetUniformLocation(phong_shader,"projection")
light_pos_loc = glGetUniformLocation(phong_shader,"lightPos")
view_pos_loc = glGetUniformLocation(phong_shader,"viewPos")
light_color_loc = glGetUniformLocation(phong_shader,"lightColor")
object_color_loc = glGetUniformLocation(phong_shader,"objectColor")

glUseProgram(simple_shader)
model_simple_loc = glGetUniformLocation(simple_shader,"model")
view_simple_loc = glGetUniformLocation(simple_shader,"view")
proj_simple_loc = glGetUniformLocation(simple_shader,"projection")
color_loc = glGetUniformLocation(simple_shader,"color")

# -------- Main loop --------
while not glfw.window_should_close(window):
    glClearColor(0.1,0.1,0.1,1)
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT)

    # -------- Draw triangle --------
    glUseProgram(phong_shader)
    glUniformMatrix4fv(model_loc,1,GL_FALSE,glm.value_ptr(glm.mat4(1.0)))
    glUniformMatrix4fv(view_loc,1,GL_FALSE,glm.value_ptr(view))
    glUniformMatrix4fv(proj_loc,1,GL_FALSE,glm.value_ptr(projection))
    glUniform3f(light_pos_loc,3,3,3)
    glUniform3f(view_pos_loc,3,3,3)
    glUniform3f(light_color_loc,1,1,1)
    glUniform3f(object_color_loc,0.8,0.3,0.2)
    glBindVertexArray(triangle_VAO)
    glDrawArrays(GL_TRIANGLES,0,3)

    # -------- Draw Bézier surface --------
    if mode == 0:  # Lambertian
        glUseProgram(phong_shader)
        glUniformMatrix4fv(model_loc,1,GL_FALSE,glm.value_ptr(glm.mat4(1.0)))
        glUniformMatrix4fv(view_loc,1,GL_FALSE,glm.value_ptr(view))
        glUniformMatrix4fv(proj_loc,1,GL_FALSE,glm.value_ptr(projection))
        glUniform3f(light_pos_loc,3,3,3)
        glUniform3f(view_pos_loc,3,3,3)
        glUniform3f(light_color_loc,1,1,1)
        glUniform3f(object_color_loc,0.2,0.5,0.9)
        glBindVertexArray(surf_VAO)
        glDrawElements(GL_TRIANGLES, len(surf_indices), GL_UNSIGNED_INT, None)

    elif mode == 1:  # Normals
        glUseProgram(simple_shader)
        glUniformMatrix4fv(model_simple_loc,1,GL_FALSE,glm.value_ptr(glm.mat4(1.0)))
```

```
        glUniformMatrix4fv(view_simple_loc,1,GL_FALSE,glm.value_ptr(view))
        glUniformMatrix4fv(proj_simple_loc,1,GL_FALSE,glm.value_ptr(projection))
        glUniform3f(color_loc,0,1,1)
        glBegin(GL_LINES)
        for i in range(len(surf_points)):
            p = surf_points[i]
            n = surf_normals[i]
            glVertex3f(*p)
            glVertex3f(*(p + 0.2*n))
        glEnd()

    elif mode == 2:  # Parametric color
        glUseProgram(simple_shader)
        glUniformMatrix4fv(model_simple_loc,1,GL_FALSE,glm.value_ptr(glm.mat4(1.0)))
        glUniformMatrix4fv(view_simple_loc,1,GL_FALSE,glm.value_ptr(view))
        glUniformMatrix4fv(proj_simple_loc,1,GL_FALSE,glm.value_ptr(projection))
        glBegin(GL_POINTS)
        for i in range(len(surf_points)):
            u = (surf_points[i][0]+1)/2
            v = (surf_points[i][1]+1)/2
            glUniform3f(color_loc,u,v,0.5)
            glVertex3f(*surf_points[i])
        glEnd()

    # -------- Draw Bézier curve and control points --------
    glUseProgram(simple_shader)
    glUniformMatrix4fv(model_simple_loc,1,GL_FALSE,glm.value_ptr(glm.mat4(1.0)))
    glUniformMatrix4fv(view_simple_loc,1,GL_FALSE,glm.value_ptr(view))
    glUniformMatrix4fv(proj_simple_loc,1,GL_FALSE,glm.value_ptr(projection))

    # Curve
    glUniform3f(color_loc,1,1,0)
    glBindVertexArray(bezier_VAO)
    glDrawArrays(GL_LINE_STRIP,0,len(bezier_points))
    # Control polygon
    glUniform3f(color_loc,1,0,0)
    glBindVertexArray(polygon_VAO)
    glDrawArrays(GL_LINE_STRIP,0,len(control_points))
    # Control points
    glPointSize(10)
    glUniform3f(color_loc,0,1,0)
    glBindVertexArray(control_VAO)
    glDrawArrays(GL_POINTS,0,len(control_points))

    glfw.swap_buffers(window)
    glfw.poll_events()

glfw.terminate()
```

7. References

1. LearnOpenGL – Lighting and Transformations
2. PyOpenGL Documentation
3. Bézier Curve – Wikipedia
4. cem_yuksel – Youtube Chanel

8. Conclusion

The project successfully demonstrates:

- Interactive visualization of Bézier curves and surfaces
- Correct implementation of discretization, normals, and parametric coordinates
- Real-time rendering using Phong shading

The implementation can be extended with higher-resolution surfaces, additional lighting models, or microfacet BRDFs for more advanced rendering.